Sports Technology | Applied AI | Systems Engineering

MatchLens: Architecting an Evidence-Grounded Cricket Intelligence & Real-Time Replay Engine for Live Sports Engagement

Technical Design Document & Forward Deployed Engineering Case Study for Dream11



Engineering case study prepared for technical review

1 November 2026

This document describes the MatchLens implementation and a proposed path toward live sports and fantasy-platform integration. It is an independent engineering case study and is not an official Dream11 publication or an assertion of affiliation.

Contents

1 Executive Summary and Forward Deployed Engineering Vision **1**
1.1 Problem statement . . . 1
1.2 Solution overview . . . 1
1.3 Current implementation boundary . . . 1
1.4 Forward Deployed Engineering (FDE) thesis . . . 2
1.5 Design principles . . . 2

2 Mathematical Formulations and Live Telemetry Engine **3**
2.1 Event model and indexing semantics . . . 3
2.2 Current run rate (CRR) . . . 3
2.3 Required run rate (RRR) . . . 3
2.4 Cricket over notation . . . 4
2.5 Batter and bowler figures . . . 4
2.6 Partnership computation . . . 4
2.7 Leverage Index: explicit proposed definition . . . 5
2.8 Verified replay checkpoints . . . 5
2.9 Replay engine invariants and test strategy . . . 6

3 Dual-Source RAG and Vector-Space Architecture **6**
3.1 Knowledge sources . . . 6
3.2 Dense embedding pipeline . . . 7
3.3 Cosine similarity . . . 7
3.4 HNSW indexing in Qdrant . . . 7
3.5 Payload and provenance model . . . 7
3.6 Dual-source retrieval . . . 8
3.7 Grounding and citation contract . . . 9

4 Intent Classification and PEFT / LoRA Architecture **9**
4.1 Intent taxonomy . . . 9
4.2 Base model and classification head . . . 9
4.3 LoRA decomposition . . . 9
4.4 Supplied training configuration . . . 10
4.5 Training and evaluation protocol . . . 11

5 LangGraph Multi-Agent State Machine **11**
5.1 Orchestration responsibilities . . . 11
5.2 Typed state contract . . . 11
5.3 Node responsibilities and transitions . . . 12
5.4 Conditional routing and failure handling . . . 13

5.5 Grounded generation contract 13

6 High-Density Frontend and Mobile Responsiveness 14

6.1 Frontend architecture 14

6.2 Information hierarchy 15

6.3 Responsive viewport containment 15

6.4 URL persistence and navigability 15

6.5 Frontend–backend contract 16

7 Production Deployment and Reliability 16

7.1 Deployment topology 16

7.2 Render backend configuration 17

7.3 Netlify frontend configuration 17

7.4 Memory and startup optimisation 18

7.5 Reliability and observability 18

7.6 API surface evidenced by the supplied backend code 18

7.7 Correctness note for catch-up intervals 19

7.8 Reliability targets to establish before scale 19

8 Dream11 Integration Roadmap 20

8.1 Integration principle 20

8.2 Phase 0: Harden the replay and evidence foundation 20

8.3 Phase 1: Connect an approved live match-data feed 20

8.4 Phase 2: Fantasy-context enrichment 20

8.5 Phase 3: Live win-probability model 21

8.6 Phase 4: Micro-engagement and platform operations 21

8.7 Integration architecture and success metrics 21

A Appendix A: Implementation Evidence and Scope Ledger 21

B Appendix B: Reproducibility and Submission Checklist 22

C Appendix C: Figure Asset Manifest 23

List of Figures

1 MatchLens end-to-end system architecture. The current data plane replays historical Cricsheet fixtures; live ingestion is a proposed extension. 2

2 Application screenshot placeholder: scorecard and live-equation presentation at a selected replay state. 6

3 Dual-source retrieval architecture. Deterministic replay telemetry and semantic rule retrieval are merged only after each source has been independently resolved. 8

4 PEFT / LoRA adapter decomposition for a transformer projection layer. 10

5 LangGraph-oriented state machine and conditional evidence-gating workflow. 13
6 End-to-end query and replay sequence with evidence retrieval. 14
7 Application screenshot placeholder: MatchLens match hub. 16
8 Application screenshot placeholder: grounded chat and Match Evidence Panel. 16

1 Executive Summary and Forward Deployed Engineering Vision

1.1 Problem statement

Live cricket interfaces are excellent at exposing the current score, but a score alone does not explain how the match reached that state. A fan entering an innings midway through a chase may need to understand the sequence behind a score, the acceleration or deterioration of the required run rate, the value of wickets in hand, or the effect of a specific delivery. A second class of questions concerns cricket laws and their application: for example, no-ball and Free Hit protocols (MCC Law 21), wide-ball decisions (Law 22), and leg-before-wicket decisions (Law 36).

A general-purpose language model is not a reliable source of scorecard arithmetic. It may produce plausible but incorrect totals, confuse legal balls with delivery events, or overstate what a law says. MatchLens addresses this failure mode by separating deterministic match computation from semantic retrieval and natural-language synthesis.

1.2 Solution overview

MatchLens is an AI-assisted cricket intelligence companion built around three cooperating capabilities:

1. **Deterministic replay telemetry.** Historical ball-by-ball data is parsed into a reproducible event sequence. A state can be reconstructed at a selected delivery index, including score, wickets, legal balls, batter and bowler figures, and chase equation.
2. **Domain knowledge retrieval.** A curated cricket-rules and T20-analytics corpus is embedded with `sentence-transformers/all-MiniLM-L6-v2` and indexed in Qdrant using 384-dimensional vectors, cosine distance, and HNSW approximate-nearest-neighbour search.
3. **Intent-aware answer orchestration.** A domain intent classifier and a LangGraph-oriented workflow direct the query to deterministic match data, semantic rule evidence, contextual statistics, and a grounded answer generator powered by Gemini 2.5 Flash.

The guiding engineering principle is simple: **a language model may explain verified evidence, but it must not invent the evidence or calculate the scorecard.** Numerical values originate from the replay engine; rules explanations are grounded in retrieved corpus passages; the generator combines them into a readable response.

1.3 Current implementation boundary

The currently shared backend code reports `mode="historical_replay"` from `/api/health`. The project notes describe 12 historical IPL matches sourced from Cricsheet. Accordingly, this document treats MatchLens as a **historical replay prototype with AI-assisted interpretation**, not as a production live-feed ingestion service. Live provider integration, stream processing, fan-scale traffic management, and Dream11 contest integration are future engineering phases.

The route excerpt supplied for this document confirms the following backend surface: health checks; match listing and metadata; state and timeline lookup; replay advancement; catch-up summaries; chat/ask; insights; and generated match stories. The implementation details of the React components, exact LangGraph graph edges, deployment telemetry, and trained classifier evaluation were not independently available in the supplied material. Those items are described as architecture or recommendations where appropriate, rather than represented as independently verified measurements.

1.4 Forward Deployed Engineering (FDE) thesis

A Forward Deployed Engineer translates a domain problem into a deployed, measurable system under real customer constraints. MatchLens demonstrates the core pattern:

- **Domain translation:** encode cricket scoring rules, over semantics, chase equations, and match context as explicit software contracts.
- **Data correctness:** derive the current state from the event ledger rather than from a model-generated summary.
- **Applied AI:** use classification, vector retrieval, and grounded generation only where they add value.
- **Deployment pragmatism:** reduce import-time memory pressure, keep API and UI independently deployable, and expose dependency health.
- **Product thinking:** convert raw telemetry into catch-up summaries, stories, contextual explanations, and a future path to fantasy engagement.

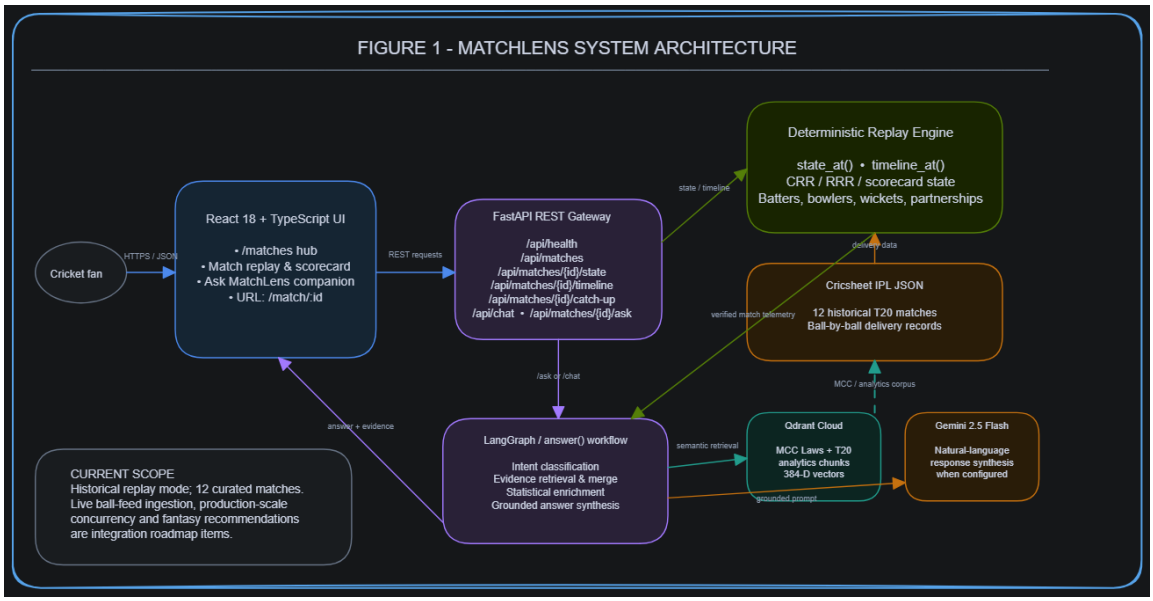


Figure 1: MatchLens end-to-end system architecture. The current data plane replays historical Cricsheet fixtures; live ingestion is a proposed extension.

1.5 Design principles

Determinism
: Score, ball count, wickets, and player figures must be reproducible from the event ledger.

Evidence provenance
: Each generated statement should be traceable to a delivery, a retrieved document, or an explicitly labelled calculation.

Graceful degradation
: A model or vector-store outage must not disable deterministic replay and scorecard access.

Bounded claims
: A missing probability model must result in an explanation of chase pressure, not a fabricated win percentage.

Operational observability
: Health, latency, dependency state, and data-version metadata should be visible to operators.

2 Mathematical Formulations and Live Telemetry Engine

2.1 Event model and indexing semantics

The replay engine treats each row in the source delivery sequence as an **event**. An event may be a legal delivery or an illegal delivery such as a wide or no-ball. Let the ordered event sequence be

$$E = (e_1, e_2, \ldots, e_N),$$

where N is the number of delivery events loaded for the innings. A replay cursor n is an event index, bounded by $0 \le n \le N$. The state after the first n events is reconstructed by folding the event reducer over $e_1, \ldots, e_n$.

For event e_i, define:

- r_i^{bat}: runs credited to the batter;
- x_i: extras recorded for the event;
- $r_i^{\text{total}} = r_i^{\text{bat}} + x_i$: team runs attributable to the event (including any separately modelled penalty runs where present);
- $\ell_i \in \{0, 1\}$: legal-delivery indicator, where wides and no-balls are not legal balls;
- $w_i \in \{0, 1\}$: wicket indicator for a dismissal counted as a team wicket under the innings model.

At cursor n, the cumulative team score, legal-ball count, and wickets lost are

$$S_n = \sum_{i=1}^{n} r_i^{\text{total}}, \tag{1}$$

$$L_n = \sum_{i=1}^{n} \ell_i, \tag{2}$$

$$W_n = \sum_{i=1}^{n} w_i. \tag{3}$$

The implementation must preserve the distinction between n, the event index, and L_n, the number of legal balls. This distinction is essential when wides and no-balls occur. The API's `ball_index` is a replay-sequence cursor; it should not be casually interpreted as a decimal over number.

2.2 Current run rate (CRR)

For an innings with at least one legal ball, current run rate is

$$\boxed{\text{CRR}_n = \frac{6S_n}{L_n}}. \tag{4}$$

If $L_n = 0$, the rate is undefined; the UI should render a dash or an explicit not-yet-started state rather than divide by zero. Team runs include extras, while batter runs do not.

2.3 Required run rate (RRR)

Let T be the target score (the score required to win is conventionally $T - S_n$), $B_{\max}$ the innings legal-ball limit (120 for a standard 20-over innings), and

$$B_n^{\text{rem}} = \max(0, B_{\max} - L_n), \qquad R_n^{\text{rem}} = \max(0, T - S_n).$$

While the chase remains active and $B_n^{\text{rem}} > 0$,

$$\boxed{\text{RRR}_n = \frac{6R_n^{\text{rem}}}{B_n^{\text{rem}}}}. \tag{5}$$

RRR is undefined after the target is reached, after the innings has ended, or when no legal balls remain and the target is unmet. An all-out state must terminate the chase even if the nominal ball limit has not been reached. The engine should expose the termination reason (target reached, all out, overs exhausted, or match result) rather than returning a misleading numeric RRR.

2.4 Cricket over notation

Cricket over notation is a mixed representation, not a decimal fraction. Given L_n legal balls,

$$O_n = \left\lfloor \frac{L_n}{6} \right\rfloor .(L_n \bmod 6). \tag{6}$$

For example, 68 legal balls are 11.2 overs, and 118 legal balls are 19.4 overs. A wides or no-balls adds an event to the replay sequence but does not advance the legal-ball component of the over. Display and arithmetic code should retain L_n as the canonical value and format O_n only for presentation.

2.5 Batter and bowler figures

For batter j, let R_j be runs off the bat and BF_j the scorer-defined balls faced. Strike rate is

$$SR_j = \begin{cases} 100\, R_j / BF_j, & BF_j > 0, \\ 0 \text{ or N/A}, & BF_j = 0. \end{cases} \tag{7}$$

The scorer's balls-faced convention must be used consistently (in particular, a wide is not a ball faced; a no-ball treatment must match the scorecard convention and parser). For bowler k, let RC_k denote runs charged to the bowler and LB_k legal balls bowled. Economy is

$$Econ_k = \begin{cases} 6\, RC_k / LB_k, & LB_k > 0, \\ 0 \text{ or N/A}, & LB_k = 0. \end{cases} \tag{8}$$

Byes and leg-byes are not charged to the bowler; wides and no-balls are charged according to the scorer's rules while not increasing legal-ball count. Wicket attribution must exclude dismissal types that are not credited to the bowler.

2.6 Partnership computation

A partnership is a time-bounded batting stand between two active batters. Let a and b be the batters in the current stand, and let τ be the event index at which that stand began. Its runs at cursor n are

$$P_{a,b}(n) = \sum_{i=\tau+1}^{n} r_i^{\text{total}}, \tag{9}$$

where the sum includes team extras recorded during the stand. Its legal-ball count is

$$B_{a,b}(n) = \sum_{i=\tau+1}^{n} \ell_i. \tag{10}$$

On a dismissal that ends the pair, the current stand is finalised and a new stand begins when the incoming batter is established. The replay reducer should also maintain the two batter identities and strike positions after each event; a partnership total without correct strike rotation is not sufficient to reconstruct the batting pair.

2.7 Leverage Index: explicit proposed definition

The supplied route excerpt confirms score and replay telemetry, but does not expose a persisted or returned field named `leverage_index`. To make this whitepaper technically complete without claiming an unverified shipped metric, this subsection defines a **candidate, versioned MatchLens leverage metric** for future analytics. It is an interpretable pressure-change score, not a win probability and not an official cricket statistic.

For an active chase, define a bounded pressure state

$$P_n = 0.50 \min\left(\frac{\mathrm{RRR}_n}{R_{\mathrm{cap}}}, 1\right) + 0.30\left(\frac{10 - W_n^{\mathrm{rem}}}{10}\right) + 0.20\left(\frac{D_n^{(K)}}{K}\right), \tag{11}$$

where $W_n^{\mathrm{rem}} = 10 - W_n$ is wickets remaining, $D_n^{(K)}$ is the number of dot balls in the last $K = 12$ legal balls (or all available legal balls early in an innings), and R_{cap} is a configurable RRR saturation threshold (initially 20 runs per over for a T20 prototype). The coefficients are initial design weights and must be calibrated on historical data before being used for user-facing recommendations.

The signed delivery leverage is

$$\boxed{\mathrm{LI}_n^{\mathrm{signed}} = 100(P_n - P_{n-1})}, \qquad \mathrm{LI}_n^{\mathrm{magnitude}} = 100|P_n - P_{n-1}|. \tag{12}$$

A positive value means the pressure score increased for the chasing side; a negative value means it decreased. The interpretation reverses when describing the bowling side. Because P_n is a deliberately simplified heuristic, production use requires versioning, backtesting, sensitivity analysis, and clear UI disclosure. A probability claim requires a separately trained and calibrated win-probability model.

2.8 Verified replay checkpoints

The supplied verification notes include two checkpoints for the SRH–RCB historical fixture (Cricsheet match identifier `1082591`). These are useful regression fixtures for scorecard arithmetic.

Metric	After event 73	After event 123 / innings end
Event count	73 delivery events	123 delivery events
Legal balls	68	118
Over notation	11.2	19.4
Score	114/2	172/10
CRR	$114 \times 6/68 = 10.06$	$172 \times 6/118 = 8.75$
Chase equation	94 runs from 52 balls	Chase ended all out; RRR is undefined
RRR	$94 \times 6/52 = 10.85$	N/A
Outcome	Chase in progress	Sunrisers Hyderabad won by 35 runs

Table 1: Supplied regression checkpoints. The legal-ball count, rather than the event count, drives overs, CRR, and RRR.

The same supplied notes report the following player figures at event 73: TM Head 23 off 17 (strike rate 135.29), KM Jadhav 30 off 14 (strike rate 214.29), and MC Henriques 1.2 overs, 16 runs, no wickets (economy 12.00). At event 123 they report BCJ Cutting 3.4 overs, 35 runs, one wicket (economy 9.55), A Choudhary 6 off 2, and YS Chahal 3 off 7 and out. These values should be retained as golden test cases and revalidated whenever parser or reducer logic changes.

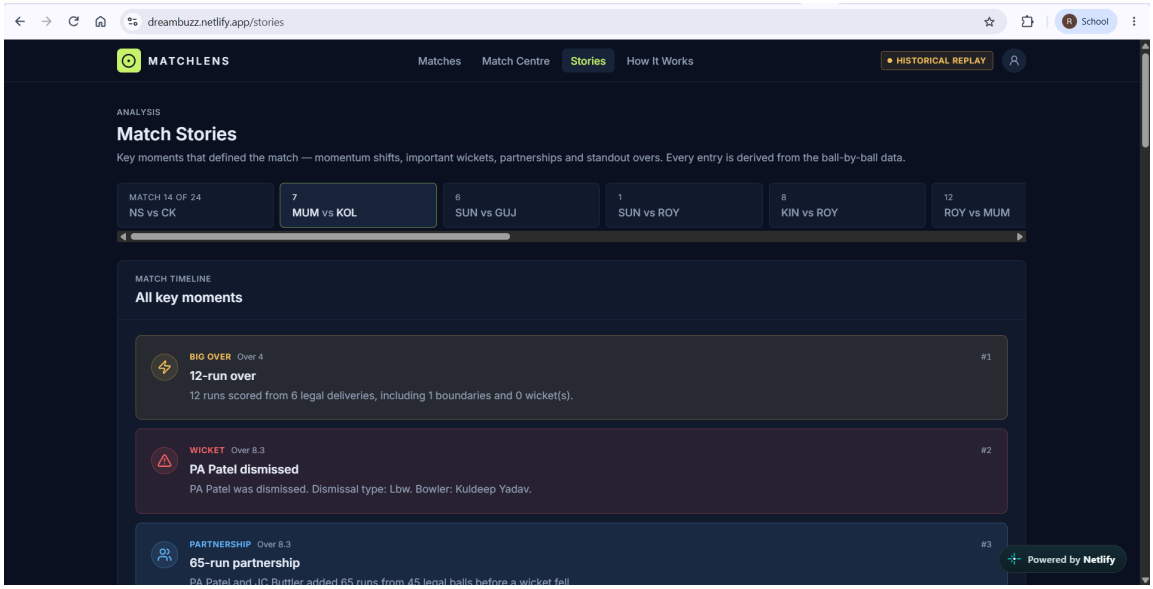


Figure 2: Application screenshot placeholder: scorecard and live-equation presentation at a selected replay state.

2.9 Replay engine invariants and test strategy

The replay reducer should satisfy the following invariants at every cursor:

1. The total score equals the sum of event-level team runs through the cursor.
2. Legal-ball count equals the number of events whose `is_legal` flag is true.
3. Wickets lost equal the count of dismissals that count toward the innings total, not simply the number of raw wicket records.
4. The formatted over string is derived from legal balls and is never parsed back into a floating-point number for arithmetic.
5. Batter and bowler aggregates reconcile with the underlying event ledger and the source scorecard.
6. A completed all-out innings cannot continue to accrue runs or expose an active chase RRR.
7. Replaying to cursor n repeatedly returns the same state (determinism and idempotence).

Property-based tests should generate sequences containing wides, no-balls, byes, leg-byes, boundaries, run-outs, and tenth-wicket dismissals. Golden-fixture tests should cover event 73 and event 123. The notes specifically identify all-out handling at event 123; that edge case should remain a first-class regression test.

3 Dual-Source RAG and Vector-Space Architecture

3.1 Knowledge sources

The project notes describe three data families:

1. **Match telemetry:** historical IPL ball-by-ball JSON from Cricsheet, including over and ball positions, batter, bowler, non-striker, runs, extras, and wicket details. The telemetry is parsed by the replay engine and is not embedded as free-form prose for arithmetic.
2. **Rules corpus:** curated excerpts from MCC Laws 21 (no-ball and Free Hit protocols), 22 (wide ball), and 36 (LBW), with source and section metadata.

3. **T20 analytics corpus:** curated explanations and formulas for chase requirements, scoring phases, wickets in hand, dot-ball pressure, and related tactical concepts.

A separate synthetic intent dataset of approximately 1,200 cricket questions is described for the six-class router. It is a classifier-training corpus, not a substitute for the rules corpus.

3.2 Dense embedding pipeline

Each rules or analytics document is normalised, divided into semantically coherent chunks, and passed through `sentence-transformers/all-MiniLM-L6-v2`. The resulting embedding is a vector in $\mathbb{R}^{384}$. For a document chunk d_i,

$$\mathbf{v}_i = f_{\text{embed}}(d_i), \qquad \mathbf{v}_i \in \mathbb{R}^{384}.$$

A user query q is embedded by the same model:

$$\mathbf{q} = f_{\text{embed}}(q).$$

The embedding model and preprocessing version must be identical between ingestion and query time. Any change to model, text normalisation, or chunking should trigger a controlled re-index rather than silently mixing incompatible vector spaces.

3.3 Cosine similarity

The relevance score between query vector $\mathbf{q}$ and chunk vector $\mathbf{v}_i$ is cosine similarity:

$$\cos(\mathbf{q}, \mathbf{v}_i) = \frac{\mathbf{q} \cdot \mathbf{v}_i}{\|\mathbf{q}\|_2 \, \|\mathbf{v}_i\|_2}. \tag{13}$$

For non-zero vectors, values closer to 1 indicate stronger directional similarity. Qdrant can be configured with cosine distance so the vector index retrieves approximate nearest neighbours according to this metric. Similarity is a retrieval signal, not a proof that a passage answers the question; returned content still requires source, section, and relevance checks.

3.4 HNSW indexing in Qdrant

Qdrant uses an HNSW (Hierarchical Navigable Small World) graph for approximate nearest-neighbour retrieval. HNSW builds navigable layers of links over vectors so search can avoid comparing a query with every vector in the collection. The main operational trade-off is between recall, query latency, memory, and index-build cost.

The project notes specify HNSW and cosine distance but do not provide concrete values for `m`, `ef_construct`, or `ef_search`. These parameters should therefore be selected using an offline retrieval benchmark rather than hard-coded as claimed project settings. The benchmark should measure recall@k against exact search, p50/p95/p99 latency, index size, and memory under representative query load.

3.5 Payload and provenance model

Each point should carry a payload with at least the fields shown below. This schema follows the supplied project notes; additional governance fields are recommended for production.

Field	Type	Purpose
`document_id`	string	Stable source-document identifier.
`title`	string	Human-readable document title.
`section`	string	Law number, clause, heading, or compendium section.
`content`	string	Text chunk returned to the answer workflow.
`source_url`	string	Provenance link to the authoritative source.
`document_type`	string	Distinguishes rule material from analytics material.
`version (recommended)`	string	Source revision or ingestion version.
`valid_from / ingested_at (recommended)`	timestamp	Supports freshness and auditability.

Table 2: Suggested Qdrant payload contract. Rows marked recommended are governance additions, not asserted fields in the current collection.

3.6 Dual-source retrieval

A question about a particular delivery may require both factual telemetry and general cricket-law knowledge. The two retrieval paths therefore remain separate until the evidence-merging stage:

- **Telemetry path:** resolve the match ID and replay cursor; retrieve the deterministic state and the relevant event window from the local match repository.
- **Semantic path:** embed the natural-language question; search the Qdrant collection; return top-ranked rule or analytics chunks with provenance.

The paths may execute concurrently, provided that the request's match ID and replay cursor are passed consistently. Their outputs are joined into a typed evidence object. If either source is unavailable, the workflow must retain the provenance boundary and avoid filling the missing source with model guesses.

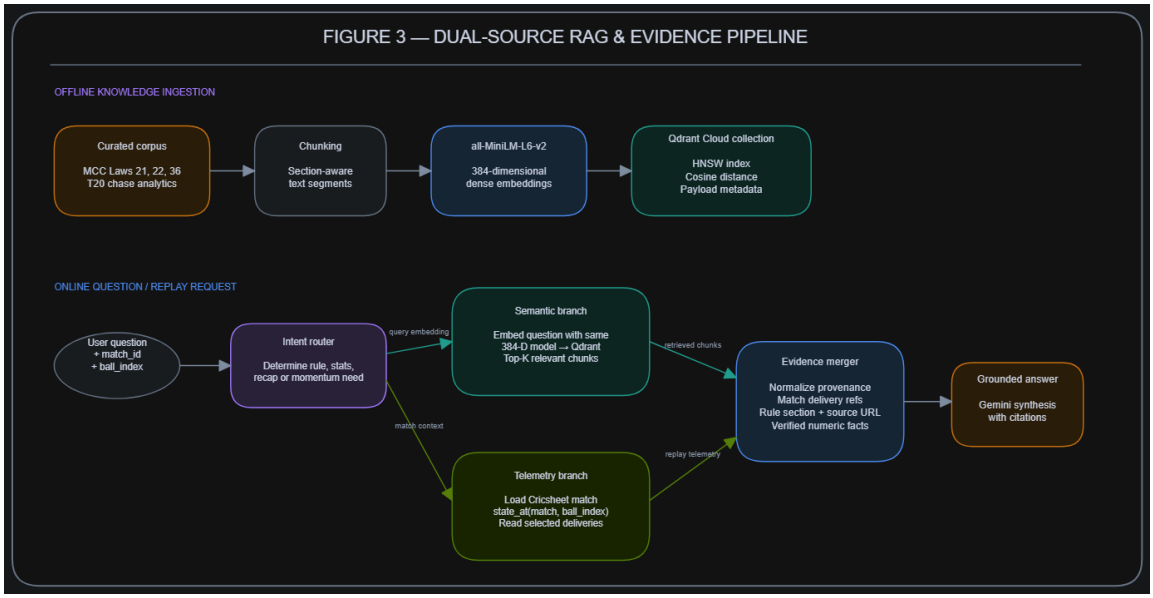


Figure 3: Dual-source retrieval architecture. Deterministic replay telemetry and semantic rule retrieval are merged only after each source has been independently resolved.

3.7 Grounding and citation contract

The response generator should receive evidence in a structured form rather than an undifferentiated prompt blob. A minimal evidence record can contain:

- match ID, event cursor, and the selected state timestamp/version;
- exact numerical values and the formula or reducer that produced them;
- relevant delivery identifiers and commentary for event-level claims;
- retrieved rule chunk, law number, section, source URL, and retrieval score;
- an explicit marker for unavailable or conflicting evidence.

The user-facing answer should distinguish delivery citations (for example, `[Delivery 11.2]`) from legal-source citations (for example, `[MCC Law 21, relevant section]`). The delivery tag should resolve to the corresponding event in the replay timeline; the law citation should resolve to the stored source metadata. A citation marker is useful only when the frontend can display the evidence it points to.

4 Intent Classification and PEFT / LoRA Architecture

4.1 Intent taxonomy

The intent router described in the project notes classifies each question into one of six labels:

1. `MATCH_STATISTICS`
2. `MATCH_RECAP`
3. `MOMENTUM_ANALYSIS`
4. `PLAYER_COMPARISON`
5. `CRICKET_RULES`
6. `GENERAL_MATCH_QUERY`

The labels are operationally useful because they can select retrieval depth and evidence requirements. For example, a statistics query should prioritise exact replay telemetry; a law query should require rule-source evidence; and a momentum query should use verified state deltas and explicitly labelled derived metrics.

4.2 Base model and classification head

The specified base encoder is `distilbert-base-uncased`. Its contextual representation is passed to a classification head that maps to six logits:

$$\mathbf{z} = W_c\mathbf{h} + \mathbf{b}_c, \qquad \hat{y} = \operatorname{argmax}_{c\in\{1,\dots,6\}} \operatorname{softmax}(\mathbf{z})_c.$$

The project notes describe a PEFT/LoRA configuration targeting the self-attention projection modules `q_lin` and `v_lin`. The classification head and any deliberately unfrozen layers must be counted separately when reporting the trainable-parameter ratio.

4.3 LoRA decomposition

Let the frozen base linear weight be $W_0 \in \mathbb{R}^{d\times k}$. LoRA learns a low-rank update using matrices

$$B \in \mathbb{R}^{d\times r}, \qquad A \in \mathbb{R}^{r\times k}, \qquad r \ll \min(d,k).$$

The adapted weight is

$$\boxed{W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA}. \tag{14}$$

The original matrix W_0 remains frozen; only A and B are trained. For one adapted linear layer, full fine-tuning updates dk weight parameters, whereas LoRA adds $r(d+k)$ trainable parameters. The ratio for that layer is

$$\rho_{\text{layer}} = \frac{r(d+k)}{dk}. \tag{15}$$

This is the parameter-efficiency advantage: when rank is small relative to the layer dimensions, the number of trainable parameters is substantially smaller than the original matrix.

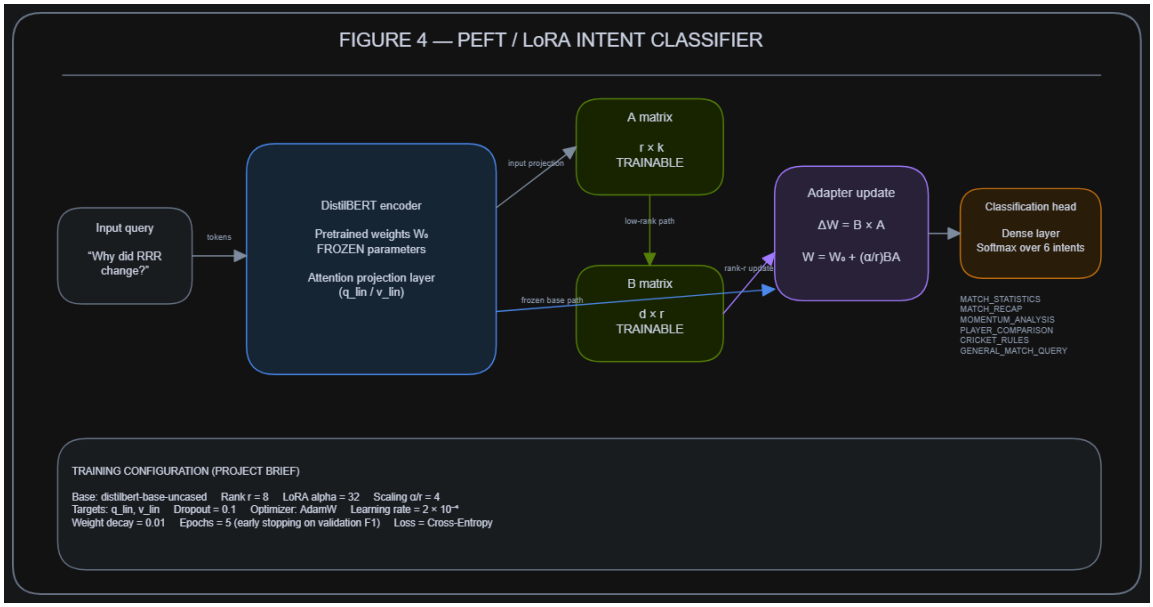


Figure 4: PEFT / LoRA adapter decomposition for a transformer projection layer.

4.4 Supplied training configuration

Hyperparameter	Specified value	Engineering note
Base encoder	`distilbert-base-uncased`	Compact encoder selected for a low-latency routing task.
LoRA rank r	8	A rank of 16 is also mentioned as an alternative experiment.
LoRA alpha α	32	For rank 8, scaling $\alpha/r = 4$.
Target modules	`q_lin`, `v_lin`	Attention query and value projections.
Dropout	0.1	Adapter dropout as specified in the project notes.
Optimizer	AdamW	Weight decay 0.01.
Learning rate	2×10^{-4}	Supplied training configuration.
Loss	Cross-entropy	Six-class single-label classification.
Epochs	5	Early stopping based on validation F1 is specified.
Training corpus	Approximately 1,200 synthetic queries	Six intent labels; class balance and split strategy should be documented.

Table 3: PEFT / LoRA configuration as supplied in the project notes.

The notes estimate that approximately 0.4%–0.7% of model parameters are trainable. The exact percentage is architecture- and configuration-dependent; it must be computed from the actual loaded model and adapter configuration rather than copied into a benchmark claim. A reproducible report should publish both counts:

$$N_{\text{trainable}} = \sum_{p:\, p.\text{requires_grad}} |p|, \qquad N_{\text{total}} = \sum_{p} |p|, \qquad \rho = 100\frac{N_{\text{trainable}}}{N_{\text{total}}}\%.$$

4.5 Training and evaluation protocol

The synthetic corpus should be split with stratification by intent, with near-duplicate query templates kept in the same split to reduce leakage. Report at minimum macro-F1, per-class precision and recall, confusion matrix, and calibration or confidence distribution. For a routing model, false negatives on `CRICKET_RULES` and `MATCH_STATISTICS` are especially important because they can cause the workflow to omit a required evidence source.

The project notes mention early stopping on validation F1 but do not include a final held-out metric, per-class scores, or measured inference latency. No accuracy or sub-20-ms performance claim is made in this report. These should be measured on the actual deployment CPU under warm and cold conditions. The fallback classifier should use explicit deterministic rules or a safe generic route when confidence is below a configurable threshold; the generator should never be asked to infer an intent by inventing unavailable data.

5 LangGraph Multi-Agent State Machine

5.1 Orchestration responsibilities

The backend tree shown in the supplied workspace includes `app/ai/graph.py` and service modules for intent classification, retrieval, match data, stories, cricket calculations, and workflow handling. The supplied API code constructs `IntentClassifier`, `CricketRetriever`, and `GeminiService`; its `/ask` and `/chat` routes delegate to an `answer(...)` workflow. This supports the architecture of an explicit orchestration layer. The exact compiled graph definition was not included in the route excerpt, so the typed state below is a design contract aligned to the described workflow, not a verbatim dump of the internal graph class.

5.2 Typed state contract

A recommended state shape is:

```python
from typing import TypedDict, NotRequired, Literal

Intent = Literal[
    "MATCH_STATISTICS", "MATCH_RECAP", "MOMENTUM_ANALYSIS",
    "PLAYER_COMPARISON", "CRICKET_RULES", "GENERAL_MATCH_QUERY"
]

class AssistantState(TypedDict):
    match_id: str
    question: str
    ball_index: int
    intent: NotRequired[Intent]
    intent_confidence: NotRequired[float]
    telemetry: NotRequired[dict]
    retrieved_rules: NotRequired[list[dict]]
    computed_statistics: NotRequired[dict]
```

```
    citations: NotRequired[list[dict]]
    answer: NotRequired[str]
    warnings: NotRequired[list[str]]
    errors: NotRequired[list[str]]
    generation_source: NotRequired[str]
```

Listing 1: Proposed typed state contract for the MatchLens query workflow.

The state should be request-scoped. It should not hold mutable global match state across users. The canonical replay cursor must be validated against the selected match's event count before a node attempts retrieval.

5.3 Node responsibilities and transitions

The intended lifecycle is:

1. **Validate request:** confirm match ID, question, and a non-negative replay cursor; load match metadata and event-count bounds.
2. **Classify intent:** run the PEFT classifier (or configured fallback) and store label and confidence.
3. **Retrieve evidence:** resolve deterministic match telemetry and, for rule or tactical questions, search Qdrant. The two branches can run concurrently.
4. **Enrich statistics:** calculate derived values such as CRR, RRR, recent dot-ball count, and any versioned leverage score from the deterministic state.
5. **Check evidence sufficiency:** verify that numerical claims have telemetry and law claims have a retrieved authoritative passage. Route insufficient evidence to an abstention or deterministic-only response.
6. **Generate grounded answer:** call Gemini 2.5 Flash with question, intent, verified telemetry, retrieved passages, calculation outputs, and citation instructions.
7. **Validate and format:** check that required citations are present and that numerical values in the answer agree with the evidence payload; return answer plus structured evidence.

5.4 Conditional routing and failure handling

Condition	Safe route	Expected behavior
Classifier unavailable or low confidence	Fallback intent / generic retrieval	Preserve match ID and cursor; do not silently label an ambiguous question as a rule query.
Qdrant unavailable	Deterministic-only path	Score, state, timeline, and numeric insights remain usable. For law questions, state that rule evidence is unavailable and abstain from a definitive ruling.
Replay data missing	Not-found response	Return the API's 404 semantics; do not substitute another match.
Gemini unavailable or times out	Deterministic response formatter	Return verified scorecard metrics and any available source excerpts; omit unsupported narrative.
Evidence conflicts	Evidence-review / abstention	Surface the conflicting source versions or values; do not choose one silently.
Citation validation fails	Regenerate once or abstain	Do not return fabricated delivery references or uncited legal claims.

Table 4: Recommended failure routing. The API health endpoint exposes Gemini, Qdrant, and classifier status; the precise runtime fallback behavior should be verified against the implementation.

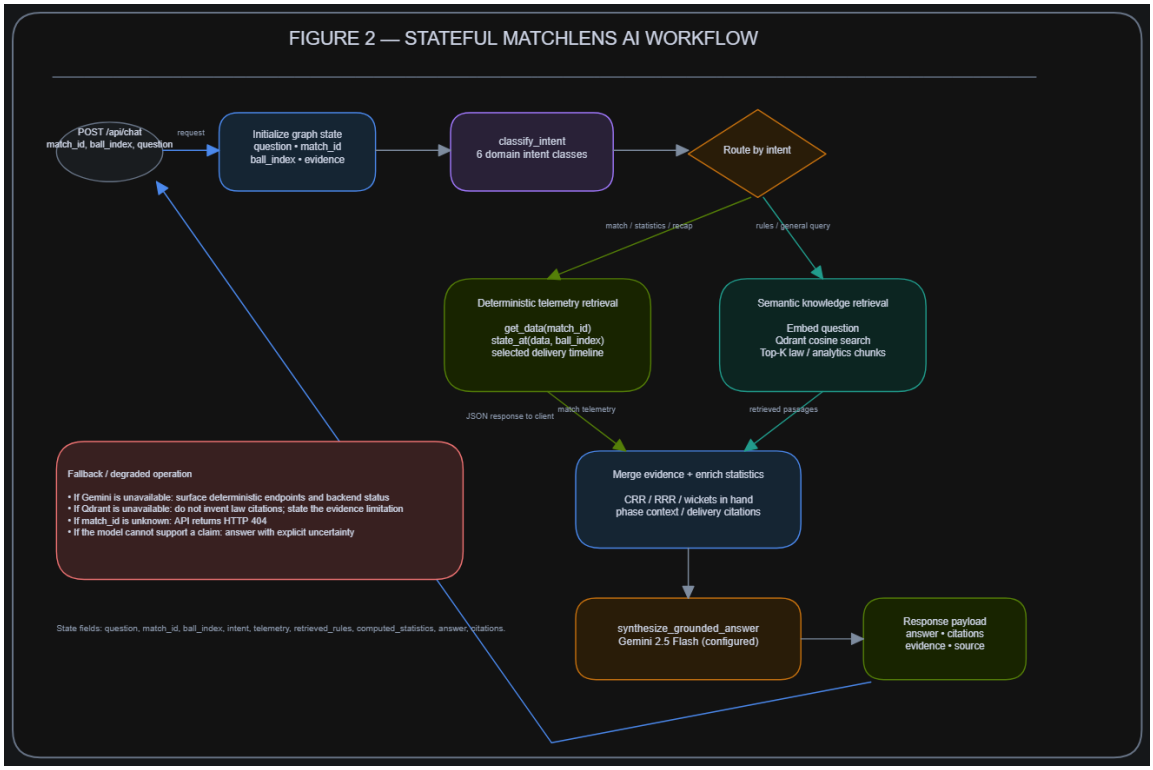


Figure 5: LangGraph-oriented state machine and conditional evidence-gating workflow.

5.5 Grounded generation contract

A system prompt for the generation node should enforce the following rules:

1. Treat the telemetry and retrieved excerpts as the only authoritative context for match-specific claims.
2. Never recalculate or replace a score, legal-ball count, target, or player figure with a guessed value.
3. Attach delivery citations to event-derived statements and law citations to retrieved legal statements.
4. Distinguish an observed statistic from a derived statistic and a heuristic interpretation.
5. Do not emit win probabilities unless a separately validated probability model has supplied the value and its calibration metadata.
6. Explicitly say when the evidence is missing, stale, or contradictory.

The answer payload should separate human-readable prose from evidence records. This allows the React client to render a Match Evidence Panel independently of the generated paragraph and enables a reviewer to inspect the exact deliveries and law excerpts supporting an answer.

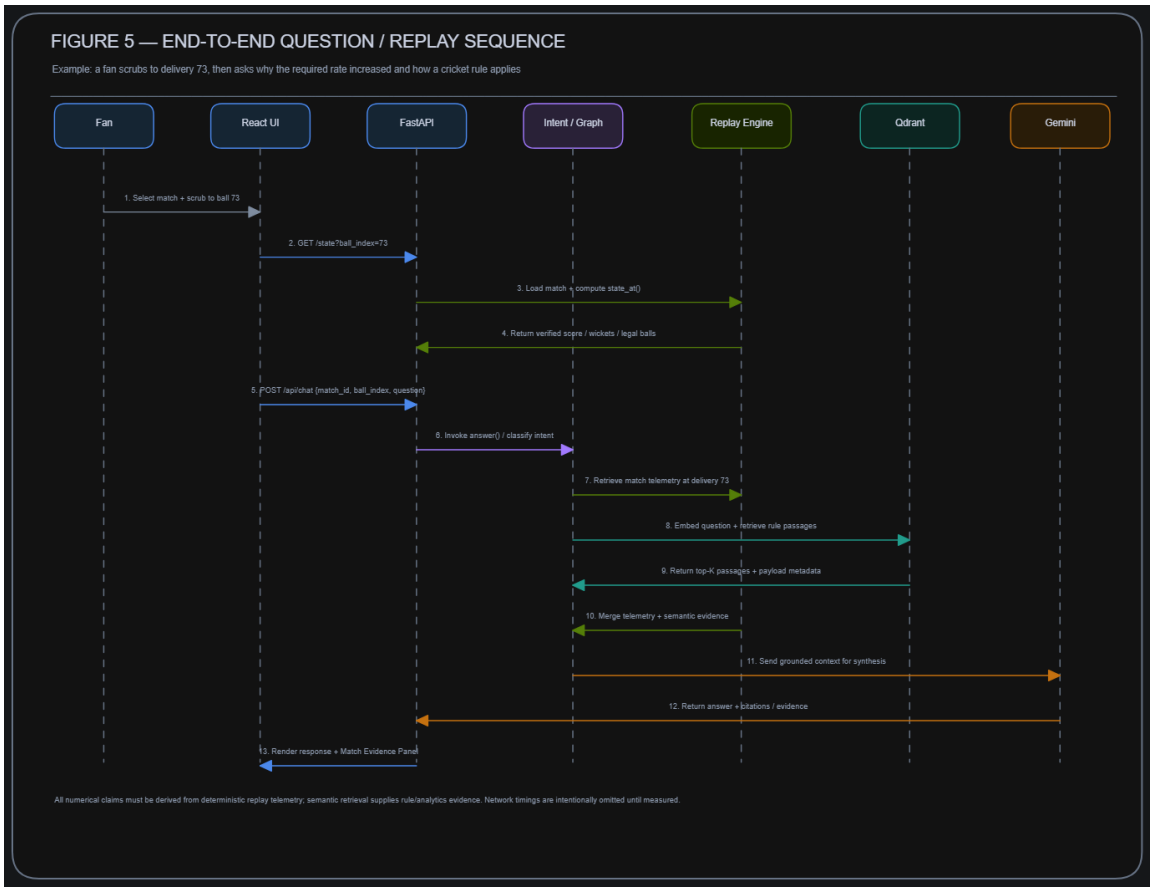


Figure 6: End-to-end query and replay sequence with evidence retrieval.

6 High-Density Frontend and Mobile Responsiveness

6.1 Frontend architecture

The project brief specifies React 18, TypeScript, Vite, Tailwind CSS, Lucide icons, Recharts, and React Router v6. The desired visual language is a Cricbuzz-inspired dark sports interface with the following palette:

Token	Hex	Purpose
Deep background	`#0B1020`	Main application canvas.
Panel background	`#111A2C`	Cards, navigation, score sections.
Electric lime	`#C7F36B`	Primary score accents and active states.
Sky blue	`#65B8FF`	Secondary information and links.
Wicket red	`#FF6868`	Wicket and warning states.

Table 5: Frontend design tokens supplied in the project brief.

6.2 Information hierarchy

A high-density sports interface should communicate the current state before presenting secondary analysis. A recommended hierarchy is:

1. Match identity, innings, score, wickets, over notation, and result/status.
2. Target, runs required, balls remaining, CRR, and RRR when a chase is active.
3. Active batting pair, striker indicator, bowler figures, and recent-over sequence.
4. Replay timeline and delivery commentary, with event-index semantics preserved.
5. Catch-up summary, match stories, insights, and Ask MatchLens evidence panel.

The supplied API supports the information architecture through endpoints for match listing, metadata, state, timeline, replay advancement, catch-up, insights, stories, and chat. A UI should load essential score state independently from AI chat so that a slow or unavailable model does not block the scoreboard.

6.3 Responsive viewport containment

The UI should avoid horizontal overflow at the document level. Recommended implementation contracts include:

- Apply `min-width: 0` to flex and grid children that contain long labels or tables.
- Use `overflow-x: auto` on scorecard table wrappers, not on the entire page.
- Define table minimum widths by content, retain readable numeric columns, and keep batter/bowler labels visible on narrow screens.
- Use `100dvh` with a safe fallback where appropriate; account for mobile browser chrome and safe-area insets.
- Keep the replay control reachable without covering the score summary or chat composer.
- Collapse secondary panels into tabs, accordions, or stacked sections at narrow breakpoints.

6.4 URL persistence and navigability

The specified routes are `/matches` and `/match/:id`. The match identifier should be encoded in the URL so that refresh, browser back/forward, and share links restore the selected fixture. If replay position is part of the intended shareable state, persist it as a query parameter (for example, `/match/1082591?ball=73`) and validate it against the event count when the page loads. This query-parameter example is a proposed extension; the project brief explicitly confirms URL persistence for the match route, not the exact cursor encoding.

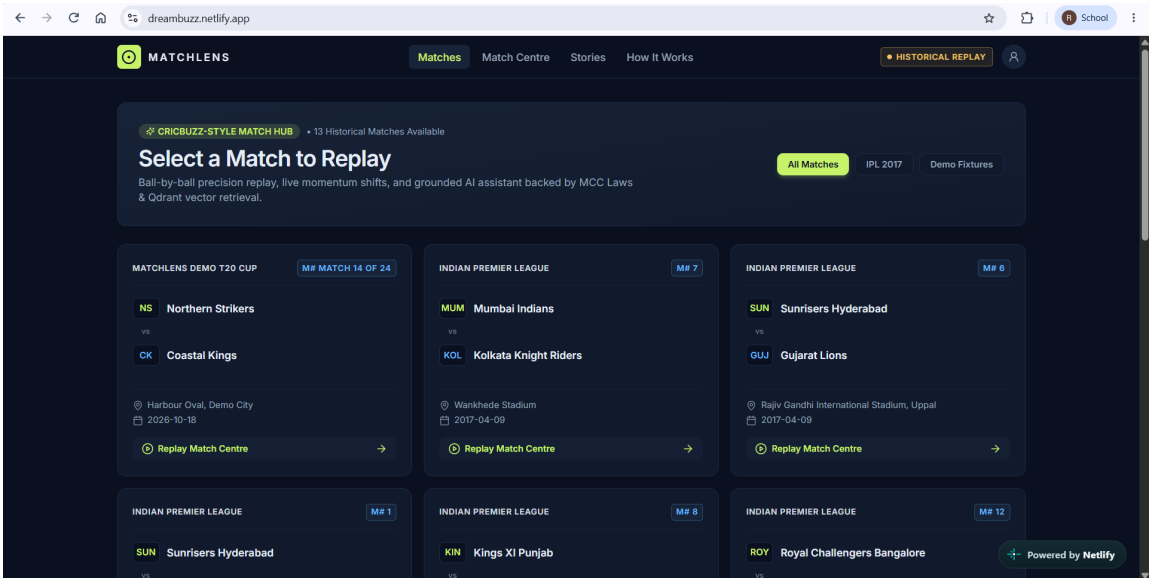


Figure 7: Application screenshot placeholder: MatchLens match hub.

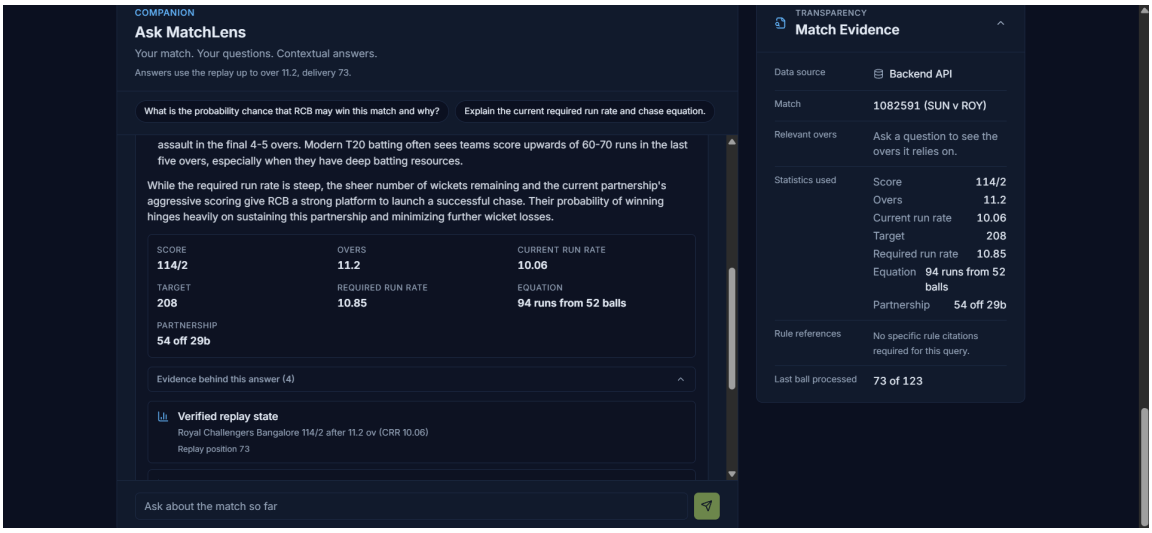


Figure 8: Application screenshot placeholder: grounded chat and Match Evidence Panel.

6.5 Frontend–backend contract

The browser should call a single configured API base URL (for example, via `VITE_API_BASE_URL`) and consume typed response models. The frontend should distinguish loading, empty, not-found, dependency-degraded, and completed-match states. Error messages should preserve backend HTTP semantics without exposing stack traces or secrets. API types should be generated or maintained from the Pydantic response models to minimise contract drift.

7 Production Deployment and Reliability

7.1 Deployment topology

The supplied deployment notes describe a split deployment:

- **Frontend:** React/Vite static assets hosted on Netlify.
- **Backend:** FastAPI application hosted as a Render Web Service, rooted at the `backend` directory.

- **External services:** Gemini API and Qdrant Cloud, configured through backend environment variables.

The frontend and backend communicate over HTTPS. The backend must allow the deployed Netlify origin through a narrowly scoped CORS policy.

7.2 Render backend configuration

The supplied deployment guide specifies the following configuration pattern:

```
# Render service root directory: backend
pip install --upgrade pip && \\
  pip install -r requirements.txt \\
  --extra-index-url https://download.pytorch.org/whl/cpu

uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Listing 2: Backend build and start commands from the supplied deployment notes.

The CPU-only PyTorch wheel index is intended to avoid pulling GPU-specific dependencies into a CPU deployment. The actual dependency resolution and image size should be checked from the Render build logs; the command alone does not establish a particular memory reduction.

The environment variables named in the supplied deployment notes are:

- `GEMINI_API_KEY`
- `GEMINI_MODEL` (specified as `gemini-2.5-flash`)
- `QDRANT_URL`
- `QDRANT_API_KEY`
- `FRONTEND_ORIGIN`

Secrets must be configured in the hosting provider's environment-variable interface and must never be committed to source control or embedded in the Vite bundle. `FRONTEND_ORIGIN` should contain the exact deployed frontend origin; a wildcard should be limited to temporary local testing, not production.

7.3 Netlify frontend configuration

The supplied deployment notes specify a root-level Vite build with `npm run build` and `dist` as the publish directory. They also report that a `netlify.toml` file was created to support React Router client-side navigation. The relevant rewrite is:

```
[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

Listing 3: Netlify single-page application fallback.

The Netlify environment variables are described as:

- `VITE_API_BASE_URL`: deployed backend base URL;
- `VITE_DEMO_MODE`: set to `false` for the backend-connected deployment.

After frontend deployment, update the backend's allowed origin to the actual Netlify site origin. Confirm that direct navigation to `/match/:id` returns the SPA shell and that API requests target the deployed backend rather than localhost.

7.4 Memory and startup optimisation

The supplied engineering notes report that eager import of PyTorch and SentenceTransformers contributed to an initial memory footprint of approximately 520 MB and that a lazy-loading refactor reduced a measured figure to approximately 45 MB. They also report use of a retriever singleton and `torch.set_num_threads(1)`. These are project-reported measurements; the measurement method, instance type, process lifecycle, and distinction between process startup and platform cold-start latency were not included in the supplied material. They should be reproduced before being used as externally audited performance claims.

The underlying strategy is sound for a constrained CPU container:

1. Keep heavyweight embedding/model imports out of the API module's initial import path where possible.
2. Initialise the retriever on first semantic query or through a controlled startup hook, not once per request.
3. Use a thread-safe singleton or application-scoped dependency so that concurrent first requests cannot create duplicate model instances.
4. Set CPU thread limits deliberately and benchmark throughput; a single-thread setting reduces contention but can also limit parallel inference capacity.
5. Keep the deterministic replay path independent of model loading.
6. Record resident set size (RSS), startup duration, model-load duration, first-query duration, and steady-state p50/p95 latency separately.

7.5 Reliability and observability

The supplied `/api/health` route returns a high-level status, the `historical_replay` mode, Gemini status/model/error details, Qdrant status, and the intent-classifier source. This is a useful service-dependency surface. Production monitoring should add:

- request counts and p50/p95/p99 latency by endpoint;
- replay-state latency and event-reconstruction errors;
- vector-search latency, retrieval-empty rate, and embedding-load failures;
- LLM timeout/error rate, token usage, and answer-validation failures;
- memory RSS, CPU, restart count, and cold/warm model initialisation timings;
- correlation IDs linking one user query to retrieval, generation, and evidence records.

7.6 API surface evidenced by the supplied backend code

The routes below are taken from the shared `api.py` excerpt. The exact JSON response schemas are defined by the project's Pydantic models and service functions; only the behavior visible in the excerpt is summarised here.

`Method`	`Path`	**Purpose visible in code**
`GET`	`/api/health`	Service mode and dependency/classifier health.
`GET`	`/api/matches`	List match repository entries.

Method	Path	Purpose visible in code
GET	/api/matches/{match_id}	Return match metadata; maps missing match to HTTP 404.
GET	/api/matches/{match_id}/state	Return reconstructed state at non-negative ball_index.
GET	/api/matches/{match_id}/timeline	Return timeline at a non-negative cursor.
POST	/api/matches/{match_id}/replay/advance	Clamp the next replay cursor to the available delivery count.
POST	/api/matches/{match_id}/catch-up	Build an interval summary and aggregate statistics.
POST	/api/matches/{match_id}/ask	Send a question to the answer workflow.
POST	/api/chat	Alias to the same question handler.
GET	/api/matches/{match_id}/insights	Return textual insights derived from the current state.
GET	/api/matches/{match_id}/stories	Build match stories from replay data.

7.7 Correctness note for catch-up intervals

The supplied `catch_up` implementation selects an interval using `start = end - request.overs * 6` over the raw `deliveries` list. Because that list includes illegal delivery events, this selects up to $6 \times$ overs *events*, not necessarily exactly the requested number of legal overs. If the product label promises the last k legal overs, the implementation should walk backwards until it has accumulated $6k$ legal balls, while retaining intervening wides and no-balls. Otherwise, the UI and API documentation should describe the interval as an event-count window. This distinction matters for an evidence-grade score summary.

Similarly, `important_events=events[-6:]` currently means the last six delivery events, not necessarily the six most important events. The key-moment list is separately assembled from wickets and batter boundaries. Naming and documentation should distinguish these fields to avoid misleading users.

7.8 Reliability targets to establish before scale

The following are proposed service-level objectives, not measured current achievements:

- Deterministic state and timeline endpoints: define a p95 latency target under a specified match size and concurrency level.
- AI answer endpoint: publish p50/p95 latency separately for warm and cold model states, and set an explicit timeout budget.
- Replay correctness: 100% pass rate on golden fixtures and invariant/property tests for every release.
- Evidence compliance: measure the proportion of match-statistic claims with valid delivery evidence and law claims with valid source citations.
- Availability: monitor backend, Qdrant, Gemini, and static frontend independently so a dependency outage is not misreported as a total application outage.

8 Dream11 Integration Roadmap

8.1 Integration principle

MatchLens is a candidate contextual-intelligence layer, not a replacement for a fantasy platform's authoritative match feed, player registry, points engine, contest service, or compliance controls. A Dream11 integration should consume trusted platform data through approved APIs and return explainable insights alongside the existing product experience. MatchLens should not independently create contest state or claim that an insight guarantees fantasy performance.

8.2 Phase 0: Harden the replay and evidence foundation

- Expand the historical fixture suite and reconcile every innings against official source scorecards.
- Add replay cursor/version identifiers, stable delivery IDs, and audit-friendly evidence records.
- Formalise the MCC source ingestion and version lifecycle; preserve source URL and section metadata.
- Evaluate classifier performance and vector retrieval quality with held-out datasets and explicit metrics.
- Add adversarial tests for wides, no-balls, run-outs, retired hurt, all-out states, super overs, and unusual innings endings where supported by the parser.

8.3 Phase 1: Connect an approved live match-data feed

Replace or complement static historical fixtures with a provider adapter that ingests official ball events. The adapter should normalise provider-specific fields into the same canonical event schema used by the replay reducer. A robust ingestion design should include:

- idempotency keys and duplicate-event detection;
- event ordering and late-arriving correction handling;
- innings and match lifecycle state transitions;
- replayable raw-event retention and parser-version metadata;
- reconciliation against provider corrections and official scorecards.

The existing replay engine can then serve both historical playback and a near-live cursor, provided the source-of-truth and correction semantics are explicit. Live ingestion is a roadmap item; it is not established by the current `historical_replay` mode.

8.4 Phase 2: Fantasy-context enrichment

With platform approval, enrich the deterministic match state with fantasy-relevant context such as player identity, eligible roles, selection/availability status, points rules, contest lock state, and platform-defined scoring events. A contextual recommendation must be explainable through inputs such as recent usage, batting position, bowling phase, venue conditions, and remaining overs. It must also respect product policy and avoid presenting speculative outcomes as certainty.

Potential experiences include:

- **Fantasy context cards:** explain why a player is gaining or losing involvement based on verified match events.

- **Captain/vice-captain context:** show role, expected opportunity window, and uncertainty drivers without guaranteeing points.
- **Rule transparency:** explain whether a dismissal or delivery event is relevant to a fantasy scoring rule, using the platform's official rules as the authority.
- **Personalised catch-up:** summarise what changed since the user's last visit, prioritised by selected players or contest context.

8.5 Phase 3: Live win-probability model

A win predictor should be a separately trained statistical or machine-learning model, not a language-model guess. Candidate inputs include score, legal balls remaining, wickets remaining, target, venue, innings phase, batting/bowling team, batter and bowler quality features where legitimately available, and historical match context. Training labels should be match outcomes, with time-aware train/validation/test splits to prevent leakage. Evaluation should include log loss, Brier score, calibration curves, and performance slices by innings phase and target range.

The model output should include a model version and calibrated probability interval or uncertainty estimate. A natural-language layer may explain the drivers using retrieved evidence, but it must not change the probability. Until this model is trained and validated, MatchLens should continue to explain the chase equation and pressure indicators rather than display a fabricated percentage.

8.6 Phase 4: Micro-engagement and platform operations

Potential engagement hooks include delivery-level explainers, wicket/boundary context cards, personalised catch-up prompts, milestone alerts, and post-match narrative summaries. Each hook should be evaluated against user trust, latency, notification fatigue, and responsible-play requirements. A/B testing should use controlled cohorts and platform-approved experimentation infrastructure.

8.7 Integration architecture and success metrics

A production integration could place MatchLens behind an internal API gateway and event adapter. The platform's official match state and points ledger remain authoritative; MatchLens supplies a contextual explanation and derived insight service. Success metrics should be agreed with product and platform teams and may include:

- time-to-understanding for users joining mid-innings;
- evidence-panel open rate and helpfulness feedback;
- reduction in repeated rule-related support queries;
- latency and availability under peak match traffic;
- factuality/citation pass rate and correction rate;
- incremental engagement measured without compromising user trust.

A Appendix A: Implementation Evidence and Scope Ledger

This appendix separates evidence visible in the supplied material from design recommendations and future work. It is intended to keep a technical submission defensible during review.

Area	Evidence supplied	Status used in this document
Historical match source	12 real historical IPL fixtures described; Cricsheet JSON schema described.	Current prototype scope as reported.
Replay calculations	Checkpoints at event 73 and event 123 with legal balls, score, CRR, RRR, batter/bowler figures, and outcome.	Supplied verified regression examples; preserve as golden tests.
Backend API	Shared `api.py` excerpt with health, matches, state, timeline, replay advance, catch-up, ask/chat, insights, and stories.	Directly evidenced by the provided code excerpt.
Health endpoint	Returns historical replay mode, Gemini status/model/error, Qdrant status, and classifier source.	Directly evidenced by the provided code excerpt.
Retriever and services	Workspace tree shows AI graph and service modules; API constructs Gemini, Qdrant retriever, and intent classifier.	Components are evidenced; internal algorithms are described from project notes.
Qdrant	384-dimensional MiniLM embeddings, cosine distance, HNSW, curated MCC laws and T20 compendium described.	Architecture as supplied; collection settings and retrieval benchmarks not supplied.
PEFT / LoRA	DistilBERT, rank 8, alpha 32, q/v modules, AdamW and six labels described.	Configuration as supplied; final evaluation metrics not supplied.
LangGraph	`app/ai/graph.py` visible in workspace tree; answer workflow invoked by API.	Orchestration architecture as described; exact graph source not supplied.
Frontend	React, TypeScript, Vite, Tailwind, Recharts, and Router v6; design tokens specified.	Stack as supplied; implementation screenshots were not supplied.
Deployment	Render + Netlify commands, variables, and SPA rewrite supplied; `netlify.toml` reportedly created.	Deployment configuration plan as supplied; public deployment status not verified.
Memory optimisation	Notes report approximately 520 MB to 45 MB after lazy loading and thread control.	Project-reported measurement; benchmark method not supplied.
Leverage Index	No exact shipped formula or API field supplied.	Explicit proposed heuristic in Section 2; not presented as implemented.
Live stream	Health endpoint explicitly returns `historical_replay`.	Live provider ingestion is roadmap, not current capability.
Dream11 fantasy features	Integration ideas supplied as desired value proposition.	Proposed roadmap only; no Dream11 integration is claimed.

B Appendix B: Reproducibility and Submission Checklist

1. Run the backend test suite and preserve the event-73 and event-123 golden outputs.
2. Verify that score, legal balls, over notation, CRR, RRR, batter figures, bowler figures, and match result reconcile with the source fixture.
3. Inspect the catch-up interval semantics when illegal deliveries appear in the requested window.
4. Record the final classifier training configuration, parameter counts, held-out metrics, and CPU inference measurements.

5. Record the Qdrant collection configuration, embedding model revision, chunking policy, retrieval benchmark, and source versions.
6. Exercise each API route with success, invalid cursor, missing match, and dependency-degraded cases.
7. Deploy the backend and frontend independently; test CORS, secrets, API base URL, and direct route refresh on Netlify.
8. Capture real screenshots for the match hub, scorecard, and companion evidence panel and export the five Excalidraw diagrams to the exact paths below.
9. Re-run PDF compilation after figures are added and inspect all page breaks, tables, links, and figure captions.

C Appendix C: Figure Asset Manifest

The document compiles with image files listed below:

1. `figures/fig1_system_architecture.png`
2. `figures/fig2_langgraph_statemachine.png`
3. `figures/fig3_qdrant_vector_pipeline.png`
4. `figures/fig4_peft_lora_diagram.png`
5. `figures/fig5_sequence_diagram.png`
6. `figures/app_screenshot_match_hub.png`
7. `figures/app_screenshot_scorecard.png`
8. `figures/app_screenshot_companion.png`

The screenshot supplied with the request shows the VS Code backend workspace and `api.py`, not the rendered application UI. It is therefore not used as a substitute for any of the three application screenshot assets above.

End of MatchLens Technical Design Document